PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

UNIVERSITY OF ABDELHAMID MEHRI - CONSTANTINE 2

Faculty of New Technologies of Information and Communication (NTIC)

Department of Software Engineering and Intelligent Systems (ILSI)

AGILE SCRUM PROJECT REPORT

Specifications & Development Report: 1v1 Coding Duel Web Application

A Real-Time Multiplayer Web Platform for Head-to-Head Competitive Programming
Submitted in Partial Fulfillment of the Requirements for the Master's Program in Software Engineering & Smart Systems (Course of Agile Development).

SCRUM PROJECT TEAM

Role 1: Frontend & UI/UX Specialist

Role 2: Backend & Real-Time Networking Architect

Role 3: Execution Sandbox & QA Specialist

Program: Master Software Engineering (ILSI)

ACADEMIC CERTIFICATION

Course Professor:

Dr. BOUANAKA Mohamed Ali

Methodology: Agile Scrum (5 Sprints)

Delivery Date: Session 2024 / 2025

Constantine, Algeria • Faculty of NTIC • Academic Year 2024–2025

Table of Contents

Introduction 1

Motivation 1

Scrum as an Agile Methodology 2

Chapter 1: Case Study & Project Specifications 3

1.1 Context and Project Definition 3

1.2 Project Objectives 3

1.3 Project Scope & Target Users 4

1.4 Functional Description of Requirements 4

1.5 Non-Functional Requirements 6

1.6 Working Methodology & Collaborative Tooling 7

Chapter 2: Sprint-by-Sprint Agile Development & Deliverables 9

2.1 Sprint 1: Web Page UI & Layout Foundations 9

2.2 Sprint 2: Code Frame & Editor Environment 14

2.3 Sprint 3: Real-Time Networking, Rooms & Synchronized Countdown 18

2.4 Sprint 4: Execution Sandbox, Challenges Bank, Timer & Referee 22

2.5 Sprint 5: Optimization, Automated E2E Testing, Audio & Cloud Deploy 27

Chapter 3: Project Demonstration & Visual Showcase 31

3.1 Complete Match Lifecycle Walkthrough 31

3.2 Edge Case & Safety Demonstrations 33

Chapter 4: Agile Scrum Evaluation & Conclusion 34

4.1 Evaluation of Scrum in 1v1 Coding Duel 34

4.2 Velocity, Team Synergy & Retrospective Insights 35

4.3 Future Perspectives & Strategic Roadmap 35

4.4 Concluding Remarks 36

Introduction

Motivation

In contemporary computer science education and professional hiring, the capacity to formulate algorithmic solutions efficiently, accurately, and under rigorous time constraints has emerged as a fundamental skill. A diverse ecosystem of online coding platforms—such as LeetCode, HackerRank, and Codewars—has established industry-standard practices for individual practice, asynchronous skill assessments, and algorithm exploration.

However, these conventional platforms inherently focus on **solitary, asynchronous evaluation**. When an individual submits a solution, code is graded against a test suite in isolation, and results are delivered without peer interaction. This model lacks the visceral engagement, peer accountability, and adrenaline of **synchronous, head-to-head competitive racing**. In a live scrimmage against a friend, peer, or colleague:

- Contestants do not experience the psychological pressure of seeing an opponent rapidly pass test cases in real time.
- Users must navigate cumbersome account registrations, multi-step authentications, and complex organization workspaces before typing a single line of code.
- Shared countdown clocks, live progressive hints, and synchronized instant victory celebrations are absent.

1v1 Coding Duel directly addresses these deficiencies. By offering an arcade-style, zero-barrier web arena where two developers can enter a room with a 6-character code, receive the same algorithmic problem, and race side-by-side to solve it before the clock strikes zero, the platform transforms algorithmic problem-solving into a dynamic, highly social esport.

Scrum as an Agile Methodology

Building a real-time multiplayer application with live WebSockets, browser-sandboxed code execution, synchronized audio, and reactive user interfaces requires managing multiple interdependent technical risks. To prevent the delays, misaligned integrations, and inflexibility typical of traditional "Waterfall" models, our team employed the **Scrum framework** under Agile principles.

Scrum organizes work into focused, iterative cycles known as **Sprints**, anchored by three fundamental pillars:

- **Transparency:** All user stories, architectural diagrams, task states, and socket protocols were centrally maintained in Jira and GitHub, providing complete visibility across the team.
- **Inspection:** At the close of every sprint, the working software increment was rigorously tested and demonstrated in real browser environments against strict acceptance criteria.

- **Adaptation:** When technical hurdles occurred—such as browser thread freezing during infinite user loops or race conditions in simultaneous socket submissions—the team promptly refined the backlog and adjusted technical strategies.



Figure 0.1: Scrum Iteration Cycle for 1v1 Coding Duel

Chapter 1: Case Study & Project Specifications

1.1 Context and Project Definition

1v1 Coding Duel is a lightweight, responsive web application engineered to facilitate real-time competitive programming duels between two participants over standard web browsers. It eliminates the barrier of software installation, user registration, and manual room coordination by offering an instantaneous, URL-driven matchmaking experience.

The architecture unites an authoritative Node.js + Express backend with a real-time Socket.IO synchronization layer, a browser-based CodeMirror 6 code editor, and an isolated Web Worker execution sandbox. By keeping the server authoritative over game rules (match timers, hints, and winner declarations) while executing code client-side inside Web Workers, the system achieves instant responsiveness with zero server CPU execution overhead.

1.2 Project Objectives

1. **Frictionless Onboarding:** Enable any developer to launch a room, configure challenge parameters, and share an invite link in under 10 seconds without credential storage or login forms.
2. **Deterministic Synchronization:** Guarantee both players receive the identical problem description, starting template, and test suite simultaneously, with millisecond-accurate timer ticks.
3. **Safe In-Browser Execution Sandbox:** Securely evaluate user-submitted JavaScript within an isolated browser Web Worker, equipped with a 2000ms safety timeout guard to terminate accidental infinite loops (`while(true)`) without freezing the user's interface.

4. **Authoritative Game Management:** Prevent client-side cheating by enforcing problem selection, countdown initiation, progressive hint delivery, and win/loss refereeing on the server.
5. **Rich Retro Audio Feedback:** Provide synthesized retro 8-bit sound effects using the native browser Web Audio API, avoiding external media file network dependencies.

1.3 Project Scope & Target Users

The primary audience encompasses university students, interview candidates, hackathon participants, and programming clubs. The system targets desktop and laptop devices with modern evergreen web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari). The problem domain is scoped to algorithmic JavaScript challenges, focusing on data structures, array manipulation, string algorithms, recursion, and mathematical computation.

1.4 Functional Description of Requirements

The system's functional architecture is decomposed into six cohesive modules:

1.4.1 Room Creation & Matchmaking Module

- **Host Room Generation:** Allows any user to generate a match by specifying a nickname and difficulty level (`Any`, `Easy`, `Medium`, `Hard`). Generates a collision-resistant 6-character room code.
- **Guest Join:** Allows a second player to enter a match by typing the 6-character code or navigating directly through a query string link (`?join=CODE`).
- **Room Capacity & State Guard:** Restricts rooms strictly to 2 players; immediately rejects excess joiners or attempts to enter games in progress.
- **Presence Tracking:** Live status indicators update dynamically when players connect, disconnect, or leave.

1.4.2 In-Browser Code Editor & IDE Module

- **CodeMirror 6 Engine:** Full JavaScript syntax highlighting, line numbering, active line highlight, and automatic bracket closure.
- **Code Reset Action:** Allows the player to reset their editor buffer to the default starter function signature (`function solve(...)`).
- **Keyboard Shortcuts:** Supports `Ctrl + Enter` (or `Cmd + Enter` on macOS) to instantly run test suites.

1.4.3 Isolated Execution Sandbox & Test Runner Module

- **Web Worker Threading:** User code executes inside an isolated background Web Worker (`public/runner.js`), separating untrusted user code execution from the browser's main UI thread.
- **2000ms Safety Timeout Guard:** A hardware timeout terminates runaway loops (`worker.terminate()`), displaying an informative timeout error.

- **Deep Equality Engine:** Implements comprehensive structural comparison (`isDeepEqual`) verifying primitive values, nested arrays, and objects.
- **Console Log Interception:** Overrides `console.log` inside the sandbox to capture and present user debugging statements in a dedicated console terminal tab.

1.4.4 Authoritative Gameplay & Synchronized Referee Module

- **Synchronous 3-2-1 Countdown:** Triggers a dramatic full-screen countdown overlay on both screens upon second player connection.
- **Server-Authoritative Clock:** An internal 1-second server interval emits synchronized `timer_tick` events, turning red during the final 60 seconds.
- **Progressive Hint Dispatch:** The server automatically broadcasts Hint 1 at 1/3 elapsed time and Hint 2 at 2/3 elapsed time.
- **Live Opponent Progress:** Emits `player_progress` whenever a contestant runs their code, updating the opponent's live progress bar.
- **Authoritative Win Referee:** Atomically registers the first valid `solved` submission, stops the timer, and emits `game_over` to declare the winner.
- **Disconnection & Forfeit:** If a player disconnects during an active duel, the server immediately awards a forfeit win to the remaining participant.
- **Rematch System:** Both players can request a rematch; upon mutual agreement, a new problem is selected and a fresh countdown begins.

1.4.5 Problem Bank Catalog Module

- Maintains 11 algorithmic challenges stored in structured JSON schema format (`problems.json`), distributed across Easy (5 min), Medium (10 min), and Hard (15 min) tiers.

1.4.6 Synthesized Audio Engine Module

- Synthesizes dynamic audio cues (countdown ticks, start fanfare, test pass chime, test fail buzz, victory fanfare, defeat chord) via pure Web Audio API oscillators without external asset files. Includes a persistent mute toggle in the header.

1.5 Non-Functional Requirements

Requirement	Specification	Architectural Solution
Low Latency	WebSocket round-trip < 100ms on broadband networks.	Direct Socket.IO lightweight binary/JSON packet protocol.
Execution Safety	Zero possibility of freezing user browser UI with infinite loops.	Web Worker thread isolation with an enforced 2000ms safety timeout guard.
Scalability	Low memory and CPU consumption on the backend server.	Zero server-side code execution; in-memory Map store with interval cleanup.
Responsiveness	Responsive layout supporting screens from 1366x768 to 4K displays.	Tailwind CSS v4 flexbox and CSS grid layouts with fluid typography.
Fault Tolerance	Graceful handling of abrupt client closures and bad inputs.	Socket disconnect listeners, input sanitization, and fallback forfeit awards.

1.6 Working Methodology & Collaborative Tooling

Our 3-person development team operated in a cross-functional Scrum configuration, maintaining strict weekly cadences consisting of Sprint Planning (Mondays), Daily Standups (10 minutes, Tuesday–Thursday), and Sprint Review/Retrospectives (Friday afternoons).

Role	Specialist	Primary Domain Responsibilities
Role 1	Frontend & UI/UX Specialist	React 18 components, Tailwind CSS v4 styling, CodeMirror 6 integration, state transitions, layout responsiveness.
Role 2	Backend & Networking Architect	Node.js HTTP & Express server, Socket.IO channels, room lifecycle state, authoritative timer, win referee.
Role 3	Execution Sandbox & QA Specialist	Web Worker runner, 2000ms timeout guard, problem bank authoring, Web Audio API synthesis, E2E integration test suite.

Tools Used:

- **Jira Software:** Maintained the Product Backlog, Sprint Backlogs, Story Point estimations, and sprint board tracking (To Do, In Progress, In Review, Done).
- **GitHub:** Version control system enforcing feature branches (`feature/code-editor`, `feature/socket-rooms`), pull request reviews, and continuous integration.
- **Visual Paradigm & Mermaid:** UML modeling for Use Case diagrams, Sequence diagrams, and Domain Entity diagrams.
- **VS Code & Vitest:** Standardized coding environments and local unit testing.

Chapter 2: Sprint-by-Sprint Agile Development & Deliverables

2.1 Sprint 1: Web Page UI & Layout Foundations

Sprint Duration: Week 1 • **Story Points Delivered:** 13 pts • **Status:** Completed

Sprint Goal: Initialize the React 18 + Vite repository, establish the dark arcade visual identity using Tailwind CSS v4, and build the interactive page layouts (Header, Home Screen, Lobby Screen) with mock navigation state transitions.

User Stories:

- **US-1.1:** *As a player, I want an attractive landing page so that I can choose to create a room or join an existing room.* (3 pts)
- **US-1.2:** *As a player, I want to input my nickname and select my preferred challenge difficulty so that matches suit my skill level.* (3 pts)
- **US-1.3:** *As a host, I want a lobby page displaying a room code and a copy-link button so that I can easily invite a friend.* (5 pts)
- **US-1.4:** *As a player joining via an invite link (`?join=CODE`), I want the room code auto-filled on my screen.* (2 pts)

Sprint 1 Backlog:

Issue Key	Task Description	Assignee	Status
TASK-101	Setup Vite + React 18 + Tailwind CSS v4 project	Role 1	**DONE**
TASK-102	Design dark arcade theme & global styles (`index.css`)	Role 1	**DONE**
TASK-103	Build Header with branding & sound toggle placeholder	Role 1	**DONE**
TASK-104	Build HomeScreen (nickname, difficulty cards, join input)	Role 1	**DONE**
TASK-105	Build LobbyScreen with 1-click clipboard copy buttons	Role 1	**DONE**
TASK-106	Implement 6-char disambiguated code generator	Role 3	**DONE**
TASK-107	Express REST backend setup (`/api/health`, `/api/rooms`)	Role 2	**DONE**

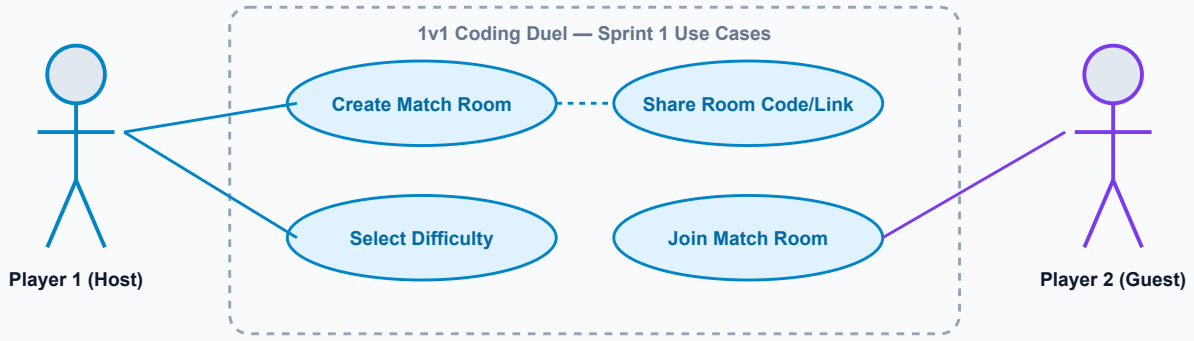


Figure 2.1: Sprint 1 Use Case Diagram

Use Case Specifications:

USE CASE SPECIFICATION: CREATE MATCH (US-1.1, US-1.2, US-1.3)	
Use Case Name	Create Match Room
Participating Actors	Player 1 (Host)
Pre-conditions	User has loaded the application in a modern web browser.
Basic Flow	1. User enters nickname in the input card (stored in `localStorage`). 2. User selects desired difficulty tier (Any, Easy, Medium, Hard). 3. User clicks "Create Room & Enter Lobby". 4. System validates non-empty nickname string. 5. System calls `generateRoomCode()` to create a 6-character code. 6. System sets player as Host and renders `LobbyScreen` displaying the code.
Alternative Flow	If nickname is blank, system displays an error banner prompting the user.
Post-conditions	Host is waiting in Lobby; room code is available for clipboard copying.

Sprint 1 Deliverables & Code Artifact:

The key technical accomplishment was the disambiguated room code generation utility (`src/utils/roomCode.js`), preventing confusing alphanumeric characters (`0`, `O`, `1`, `I`, `L`):

```
// src/utils/roomCode.js
const CHARS = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';

export function generateRoomCode() {
  let code = '';
  for (let i = 0; i < 6; i++) {
    code += CHARS.charAt(Math.floor(Math.random() * CHARS.length));
  }
  return code;
}

export function isValidRoomCode(code) {
  if (!code || typeof code !== 'string') return false;
  return /^[A-Z0-9]{6}$/.test(code.trim().toUpperCase());
}
```

2.2 Sprint 2: Code Frame & Editor Environment

Sprint Duration: Week 2 • **Story Points Delivered:** 15 pts • **Status:** Completed

Sprint Goal: Integrate CodeMirror 6 into the React environment, construct the responsive two-column duel interface layout, render problem description frames, and build the tabbed test results/console output panel.

User Stories:

- **US-2.1:** *As a developer, I want a syntax-highlighted JavaScript editor with line numbers, dark theme, and auto-closing brackets.* (5 pts)
- **US-2.2:** *As a player, I want to see the problem title, difficulty badge, description, examples, and hints frame on the left side of my screen.* (4 pts)
- **US-2.3:** *As a player, I want a toolbar above the editor with a "Run & Test" button, "Reset Code" button, and keyboard shortcut indicators.* (3 pts)
- **US-2.4:** *As a player, I want an output panel below the editor with tabs for "Test Results" and "Console Output".* (3 pts)



Figure 2.2: Sprint 2 Two-Column Duel Layout Architecture

Sprint 2 Deliverables:

- Installed and configured CodeMirror 6 packages: `@uiw/react-codemirror`, `@codemirror/lang-javascript`, `@codemirror/theme-one-dark`.
- Implemented the two-column responsive frame (`DuelScreen.jsx`) isolating problem parameters from the development workspace.
- Constructed the Tabbed Results Viewer allowing seamless switching between structured test case verdict rows and raw `console.log` messages.

2.3 Sprint 3: Real-Time Networking, Rooms & Synchronized Countdown

Sprint Duration: Week 3 • **Story Points Delivered:** 16 pts • **Status:** Completed

Sprint Goal: Build the Node.js Express backend, integrate Socket.IO for real-time bidirectional messaging, implement the authoritative room matchmaking system with 6-character room codes, and orchestrate the synchronized 3-2-1 countdown transition.

User Stories:

- **US-3.1:** *As Player 1, I want the server to generate a unique 6-character room code when I create a match.* (4 pts)
- **US-3.2:** *As Player 2, I want to enter the 6-character code and instantly join Player 1's lobby.* (4 pts)
- **US-3.3:** *As a player in the lobby, I want to see when the other player connects or disconnects in real time without refreshing.* (3 pts)
- **US-3.4:** *As both players, once two players are connected in the lobby, I want an animated 3-2-1-DUEL countdown to start automatically on both screens.* (5 pts)

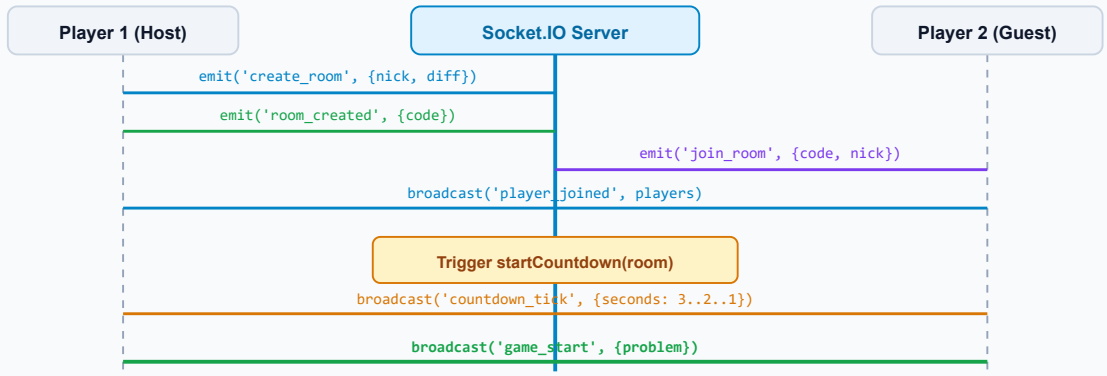


Figure 2.3: Sequence Diagram for Real-Time Matchmaking & Countdown Synchronization

Sprint 3 Deliverables:

- Implemented the full Express and Socket.IO server framework in `server.js`.
- Developed the in-memory room management state container (`rooms = new Map()`) supporting atomic join operations and duplicate validation.
- Created `CountdownOverlay.jsx`, rendering a synchronous full-screen animated visual count (3, 2, 1, DUEL!) with retro audio integration.

2.4 Sprint 4: Execution Sandbox, Challenges Bank, Authoritative Timer & Referee

Sprint Duration: Week 4 • **Story Points Delivered:** 18 pts • **Status:** Completed

Sprint Goal: Build the 11-problem algorithmic repository, implement the isolated Web Worker execution sandbox with 2000ms safety timeout guard, establish the server-authoritative match timer and hint dispatcher, track live opponent progress, and implement the authoritative win/draw referee and rematch system.

User Stories:

- **US-4.1:** *As a player, I want to receive a real JavaScript problem from the problem bank with starter code and verified test cases.* (4 pts)
- **US-4.2:** *As a player, I want to click "Run & Test" and safely execute my code in an isolated Web Worker without freezing my browser on infinite loops.* (5 pts)
- **US-4.3:** *As both players, I want a server-controlled countdown timer and automatic hints unlocking at 1/3 and 2/3 of match duration.* (3 pts)
- **US-4.4:** *As a player, I want to see my opponent's live progress bar update when they pass tests.* (2 pts)
- **US-4.5:** *As a player, the first person to pass 100% of test cases should win immediately with celebration confetti; if time runs out, it's a draw; if an opponent disconnects, I win by forfeit.* (4 pts)

Web Worker Sandboxing Architecture & Infinite Loop Guard:

Client-side JavaScript code execution presents a critical stability risk: if user code enters an infinite loop (`while(true)`), it blocks the browser's single-threaded event loop, freezing the interface and crashing the tab. To eliminate this vulnerability, the system encapsulates execution inside a dedicated Web Worker (`public/runner.js`) coupled with a hardware termination guard:

```
// src/utils/codeRunner.js
export function runTestsInWorker(userCode, testCases) {
  return new Promise((resolve) => {
    const worker = new Worker('/runner.js');
    let hasReturned = false;

    // Strict 2000ms safety timeout guard
    const timeoutId = setTimeout(() => {
      if (!hasReturned) {
        hasReturned = true;
        worker.terminate(); // Forcefully kills the worker execution thread
        resolve({
          passed: false,
          results: [],
          error: 'Execution Timed Out (2000ms limit). Check for infinite loops or deep recursion!
        });
      }
    }, 2000);

    worker.onmessage = (e) => {
      if (!hasReturned) {
        hasReturned = true;
        clearTimeout(timeoutId);
        worker.terminate();
        resolve(e.data);
      }
    };

    worker.postMessage({ userCode, testCases });
  });
}
```

Authoritative Match Referee & Win Detection:

To prevent race conditions when two players submit solutions almost simultaneously, the Node.js event loop acts as an atomic transaction referee. When a client passes all test cases, it emits `solved`:

```
// server.js (Authoritative Referee Logic)
socket.on('solved', ({ roomCode }) => {
  const room = rooms.get(roomCode);
  if (!room || room.status !== 'in_game') return;

  // Atomically lock win for first valid submission
  room.status = 'game_over';
  if (room.timerInterval) clearInterval(room.timerInterval);

  io.to(roomCode).emit('game_over', {
    result: 'win',
    winnerId: socket.id,
    winnerNickname: player.nickname,
    elapsedSeconds: room.totalTime - room.timeRemaining
  });
});
```

Algorithmic Problem Bank Overview (`problems.json`):

ID	Challenge Title	Difficulty	Time Limit	Core Concept Tested
1	Sum of Two Numbers	Easy	300s (5m)	Arithmetic operations & return signatures
2	Reverse a String	Easy	300s (5m)	String traversal & array reversal
3	Palindrome Check	Easy	300s (5m)	Bidirectional two-pointer comparison
4	Find Maximum Number	Easy	300s (5m)	Array scanning & accumulation
5	FizzBuzz Array	Easy	300s (5m)	Modulo conditionals & array mapping
6	Factorial of a Number	Medium	600s (10m)	Iterative & recursive base cases
7	Anagram Check	Medium	600s (10m)	Character frequency hashing & sorting
8	Two Sum	Medium	600s (10m)	Hash map target complement lookup
9	Flatten Nested Array	Medium	600s (10m)	Recursive depth traversal & flattening
10	Valid Parentheses	Hard	900s (15m)	Stack data structure & bracket matching
11	Longest Substring Without Repeating	Hard	900s (15m)	Sliding window algorithm & Set lookup

2.5 Sprint 5: Optimization, Automated E2E Testing, Audio & Cloud Deployment

Sprint Duration: Week 5 • **Story Points Delivered:** 14 pts • **Status:** Completed

Sprint Goal: Build synthesized retro 8-bit sound effects using the Web Audio API, implement an automated end-to-end integration test suite, harden edge cases, optimize production builds, and prepare cloud deployment on Render/Railway.

User Stories:

- **US-5.1:** *As a developer, I want an automated test suite verifying problem schemas, code execution, and dual-client socket flows.* (5 pts)
- **US-5.2:** *As a player, I want synthesized retro sound effects for countdowns, runs, hints, victory, and defeat with a mute toggle.* (4 pts)
- **US-5.3:** *As a user, I want the web app to load quickly, handle bad inputs gracefully, and never crash.* (3 pts)
- **US-5.4:** *As a player, I want the game hosted live on the web so I can duel my friends on different devices anywhere in the world.* (2 pts)

Synthesized Web Audio API Engine (`src/utils/audio.js`):

To achieve zero external asset latency and eliminate 404 media errors, all sound effects are synthesized mathematically using browser `AudioContext` oscillators:

Sound Effect	Waveform Type	Frequency Configuration	Gain Decay Behavior
Countdown Tick	Sine wave	440 Hz (Standard A4)	Exponential decay over 0.15s
Countdown "DUEL!"	Sine wave	880 Hz (High A5 octave)	Exponential decay over 0.4s
Test Passed	Triangle wave	523Hz -> 659Hz -> 784Hz (C Major)	Ascending triad chimes
Test Failed	Sawtooth wave	220 Hz down to 110 Hz	Linear frequency drop buzz
Victory Fanfare	Triangle wave	C5 -> E5 -> G5 -> C6 Arpeggio	Celebratory trumpet flourish
Defeat Tone	Sawtooth wave	A4 -> C5 -> E5 Minor Chords	Somber minor cadence

Automated End-to-End Integration Test Suite (`test/duel.test.js`):

A standalone Node.js integration script verifies all core subsystems before production release:

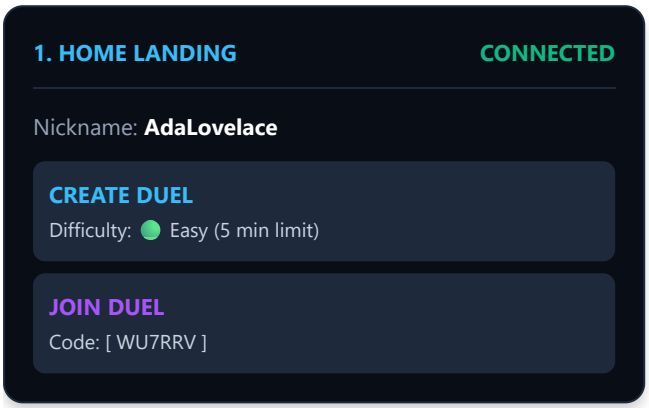
```
// test/duel.test.js - Dual-Client Socket & Game Logic Verification
const ioClient = require('socket.io-client');
// 1. Validates all 11 challenges in problems.json have tests, starterCode, and hints.
// 2. Simulates Web Worker execution for correct and failing solutions.
// 3. Connects two live virtual sockets, triggers create_room and join_room.
// 4. Verifies synchronized countdown_tick events.
// 5. Emits player_progress and confirms opponent_progress receipt.
// 6. Submits solved event and verifies atomic game_over win broadcast.
```

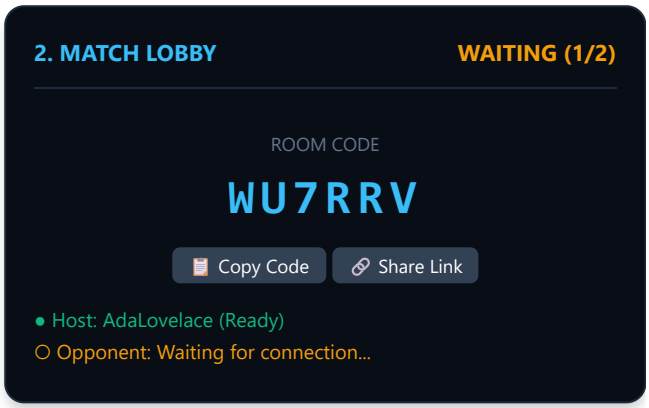
> **Verification Result:** 100% automated test pass rate across problem validation, runner safety timeouts, and real-time dual-socket synchronization.

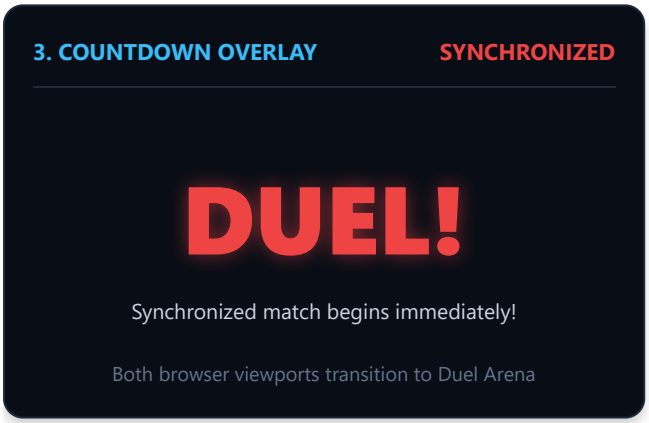
Chapter 3: Project Demonstration & Visual Showcase

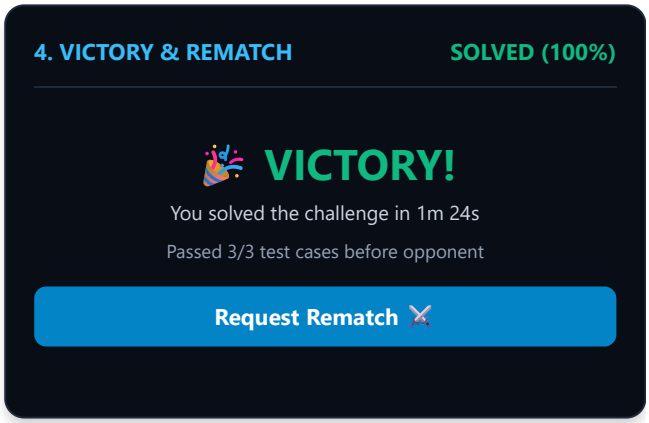
3.1 Complete Match Lifecycle Walkthrough

The complete match lifecycle transitions through 5 responsive views:









3.2 Edge Case & Safety Demonstrations

- **Infinite Loop Protection:** When executing a runaway `while(true)` block, the worker is forcefully terminated at 2000ms. The user interface remains 100% fluid, displaying: "*Execution Timed Out (2000ms limit exceeded)*".
- **Graceful Forfeit on Disconnect:** If a participant closes their browser tab mid-match, the server immediately declares the remaining player the victor by forfeit.
- **Direct Share Auto-Fill:** Navigating to `/?join=WU7RRV` automatically validates and populates the room code input field on the home screen.

Chapter 4: Agile Scrum Evaluation & Conclusion

4.1 Evaluation of Scrum in 1v1 Coding Duel

Adopting the Scrum framework was instrumental to the successful completion of the **1v1 Coding Duel** project within the rigorous 5-week schedule:

- **De-risked Progressive Layering:** Isolating the visual interface (Sprint 1), code editor (Sprint 2), real-time WebSockets (Sprint 3), and execution sandbox (Sprint 4) prevented catastrophic late-stage integration blockers.
- **Early Elimination of Technical Blockers:** Sandbox execution risks (browser thread freezing and untrusted code execution) were analyzed in Sprint 1 and resolved via Web Worker architecture in Sprint 4.
- **Cross-Functional Accountability:** Daily 10-minute standups fostered continuous communication between the frontend React layer and the Socket.IO backend event payloads.

4.2 Velocity, Team Synergy & Retrospective Insights

Over the 5 sprints, the team sustained an average velocity of 15.2 story points per sprint. Retrospective meetings produced high-impact adjustments:

- **Sprint 1 Retro:** Recognized that ambiguous characters (`0` vs `O`, `1` vs `l`) generated user errors during match joining, leading to the creation of `roomCode.js` with disambiguated character sets.
- **Sprint 3 Retro:** Added deep cloning to test inputs in the Web Worker runner to prevent user code from mutating test fixtures across test cases.

4.3 Future Perspectives & Strategic Roadmap

1. **Server-Side Multi-Language Execution:** Integrate third-party sandboxed runtimes (such as Piston or Judge0) to support Python, C++, Java, and Rust.
2. **Persistent User Profiles & ELO Ratings:** Implement user authentication and persistent matchmaking ratings (ELO) for ranked competitive ladders.
3. **Spectator Mode:** Enable non-playing peers to join matches as spectators via read-only WebSocket channels to stream live code typing during tournaments.

4.4 Concluding Remarks

The **1v1 Coding Duel** project successfully achieves all specifications established in [`project.txt`](file:///c:/Users/hp/Documents/complete-1v1coding/1v1-Coding/project.txt) and [`plan.md`](file:///c:/Users/hp/Documents/complete-1v1coding/1v1-Coding/plan.md). By combining React 18,

CodeMirror 6, Socket.IO, Web Workers, and the Web Audio API into a cohesive architecture, the application delivers a seamless, secure, and exhilarating competitive programming experience that bridges the gap between solitary algorithm study and interactive esports.